Lecture 14/15 2026-10-15 & 10-20

Today: linear quadratic regulator



Aside: LQR appears in optimal control, reinforcement learning

Today: we'll introduce <u>policies</u> and <u>cost-to-go</u>

Discrete-time linear quadratic regulator (LQR):

$$\min_{x_{0:N}, u_{0:N-1}} \quad x_N^T Q_g x_N + \sum_{k=0}^{N-1} x_k^T Q x_k + u_k^T R u_k$$

$$\text{subj. to:} \quad x_{k+1} = A x_k + B u_k \qquad k = 0, \dots, N-1$$

$$x_0 = x_{init}$$

$$x_k \in \mathbb{R}^{n_x} \qquad u_k \in \mathbb{R}^{n_u}$$

$$Q, Q_g \succeq 0 \qquad R \succ 0$$

Note: "vanilla" LQR does not include inequality constraints

Standard QP formulation would solve this with

$$\mathcal{O}(N^3 (n_x + n_u)^3) \text{ complexity}$$

(N: horizon; $n_x + n_u$: state + control)

Can we do better?

Bellman's Principle of Optimality:

"An optimal policy has the property that whatever the initial state and initial decisions are, the

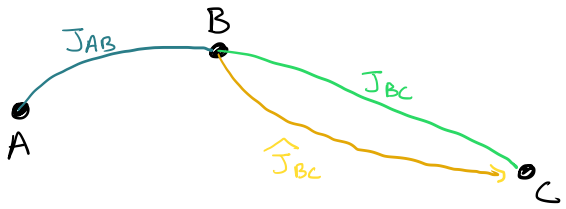
remaining decisions must constitute an optimal policy with regard to the state resulting from the first decisions."

R. Bellman, "An Introduction to the Theory of Dynamic Programming", 1953.

Today: 2026-10-20

- Final project: sign up for feedback session next week
- Quiz 3: on Tu 10/27; will cover definitions of convexity, quadratic program solution techniques, & discussions on solver/interfaces for off-the-shelf trajectory optimization
  Will not include: derivation of PDG,
  May include: linear least squares, least norm, KKT

Bellman principle: loosely "tail optimality"



Claim: Suppose the optimal cost to go from A to C corresponds to $A \to B \to C$ with cost $J^*_{AC} = J_{AB} + J_{BC}$

If $A \to B \to C$ is the optimal path from A to C, then $B \to C$ is the optimal path to go from B to C with cost J^*_{BC}

Proof-by-contradiction: suppose there were a better path

by-contradiction: suppose there were a better path $B \to C$ with cost $\hat{J}_{BC}$, then we'd have

$$\hat{J}_{BC} < J_{BC}$$
$$\downarrow$$
$$J_{AB} + \hat{J}_{BC} < J_{AB} + J_{BC}$$

But we already said $J^*_{AB} = J_{AB} + J_{BC}$, so this is a contradiction!

Takeaway: we can use DP to solve the optimal control problem by working backwards from the goal.

In general, DP scales poorly to high-dimensional state and action spaces → LQR is a special case with an analytical solution.

Cost-to-go function (also known as value function):

$$V_k(z) = \min_{u_{k:N-1}} x_N^T Q_g x_N + \sum_{t=k}^{N-1} x_t^T Q x_t + u_t^T R u_t$$
$$x_k = z$$
$$x_{t+1} = A x_t + B u_t \qquad t = k, \ldots, N-1$$

NOTE: the cost-to-go is a function of the "time" k and measures the optimal cost that can be accrued starting at state z and going from time k to N

How do we leverage Bellman?
→ We use $V_k(z)$ recursively:

$$V_k(z) = \min_{u_{k:N-1}} \; x_N^T Q_g x_N + \sum_{t=k}^{N-1} x_t^T Q x_t + u_t^T R u_t$$

$$x_k = z$$

$$x_{t+1} = A x_t + B u_t \qquad t = k, \dots, N-1$$

$$= \min_{w_k} \; z^T Q z + w_k^T R w_k + V_{k+1}(Az + Bw_k)$$

stage cost — cost-to-go at next state

How do we solve this?

1. We apply recursion with $V_k(z)$
2. We are going to use an ansatz (i.e., guess)

Ansatz: to solve this recursion problem, we assume the solution looks like:

$$V_{k+1}(z) = z^T P_{k+1} z \qquad (\text{similarly, } V_k(z) = z^T P_k z)$$

(P_{k+1} depends on k)

$$V_k(z) = \min_{w_k} \; z^T Q z + w_k^T R w_k + V_{k+1}(Az + Bw_k)$$

this is a quadratic expression

$$= \min_{w_k} \; z^T Q z + w_k^T R w_k + (Az + Bw_k)^T P_{k+1} (Az + Bw_k)$$

$\Rightarrow$ Apply N.C.O.: $\partial V_k(z)$

$\Rightarrow$ Apply N.C.O.: $\frac{\partial V_k(z)}{\partial w_k} := 0$

$\rightarrow \frac{\partial V_k(z)}{\partial w_k} = 2R w_k^* + 2B^T P_{k+1} B w_k^* + 2B^T P_{k+1} A z := 0$

$= 2(R + B^T P_{k+1} B) w_k^* + 2 B^T P_{k+1} A z = 0$

$\rightarrow 2(R + B^T P_{k+1} B) w_k^* = -2 B^T P_{k+1} A z$

$\rightarrow w_k^* = -(R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A z$

$= -K_k z$

where $K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$

NOTE: this is simply a linear controller!

$\Rightarrow$ Linear state feedback would allow for you to precompute all K_k matrices for $k = 0, \ldots, N-1$

e.g., If you had a controller operating at 1000Hz, you may not be able to run primal-dual IPM on a microcontroller but you could use LQR

$\Rightarrow$ Now plug this expression back into $V_k(z) = z^T P_k z$

$\rightarrow P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$

NOTE: P_k will be positive semidefinite by construction

What is the base case for this recursion?

What is the base case for this recursion?

$\rightarrow V_N(z) = z^T Q_g z \qquad$ so $P_N = Q_g$

Full LQR algorithm:

1. Define base case $V_N(z) = z^T Q_g z \qquad (P_N = Q_g)$

2. for $k = N-1 \rightarrow 0$

   Evaluate P_k using expression above

   Evaluate K_k gain matrix

Now given some $x_0 = x_{init}$, can compute $u_{0:N-1}$

Takeaways:

1. We've solved the optimal control problem by constructing a policy: $w_k^* = -K_k z$

   We'll return to the idea of a policy when we solve stochastic optimal control problems

   $\rightarrow$ A policy maps state z to control w

2. Reduced the complexity of the problem:

   Previously: we inverted $((n_x + n_u) \cdot N)^3$

Previously: we inverted $((n_x+n_u)\cdot N)^3$

By breaking down each time step: $\mathcal{O}(N(n_x+n_u)^3)$

↳ complexity is linear in N now